The request is received from the network by your ASP.NET Core application. Every ASP.NET Core application has a built-in web server—Kestrel, by default—that is responsible for receiving raw requests and constructing an internal representation of the data, an `HttpContext` object, which the rest of the application can use.

Your application can use the details stored in `HttpContext` to generate an appropriate response to the request, which may be to generate some HTML, to return an "access denied" message, or to send an email, all depending on your application's requirements.

When the application finishes processing the request, it returns the response to the web server. The ASP.NET Core web server converts the representation to a raw HTTP response and sends it to the network, which forwards it to the user's browser.

To the user, this process appears to be the same as for the generic HTTP request shown in figure 1.2: the user sent an HTTP request and received an HTTP response. All the differences are server-side, within your application.

You've seen how requests and responses find their way to and from an ASP.NET Core application, but I haven't yet touched on how the response is generated. Throughout this book, we'll look at the components that make up a typical ASP.NET Core application and how they fit together. A lot goes into generating a response in ASP.NET Core, typically within a fraction of a second, but over the course of the book we'll step through an application slowly, covering each of the components in detail.

1.5 What you'll learn in this book